2025 IEEE 21st International Conference on Automation Science and Engineering (CASE)
August 17-21, 2025. Los Angeles, CA, US

2025 IEEE 21st International Conference on Automation Science and Engineering (CASE) | 979-8-3315-2246-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/CASE58245.2025.11163879

Automated Tuning of Non-Differentiable Rigid Body Simulation Models for Wheeled Mobile Robots

Marvin Wiedemann[1], Ossama Ahmed[1], Mrunal Hatwar[1], Renato Gasoto[2], Peter Detzner[1], and Sören Kerner[1]

***Abstract*— Simulation plays a crucial role in robotics development, yet creating and tuning accurate models with a small sim-to-real gap remains challenging and limits their broader applicability. This work introduces a pipeline for the automated tuning of models of wheeled mobile robots for simulation tools based on non-differentiable physics engines. Leveraging real-world motion data, the pipeline computes a sim-to-real error and applies black-box optimization to refine model parameters. Three design goals are followed: flexibility in implementing different optimization algorithms, applicability to wheeled mobile robots with diverse kinematics, and the ability to tune multiple model parameters. Experiments involving four wheeled mobile robots with differential, Ackermann, and omnidirectional kinematics validate the approach across diverse trajectories and algorithms. Results indicate successful automated tuning, revealing insights into the relationships between robot complexity, trajectory dynamics, and optimization algorithms.**

I. Introduction

Simulation is a powerful tool in robotics development, enabling parallel hardware and software design, creating virtual testing and training environments, and generating synthetic data for machine learning. However, the complexity of robotic systems, with numerous components such as actuators and sensors, requires extensive modeling efforts and substantial computational power. To manage this complexity, rigid body approximations are commonly employed, although they may overlook certain physical phenomena [1]. These simplifications lead to a difference between simulation and real-world behavior, known as the sim-to-real gap [2]. This gap poses significant challenges in the transfer of developments from simulation to real systems.

Existing approaches to bridge the sim-to-real gap can be categorized into two types [3]. The first involves randomization techniques, which vary the simulation parameters to enhance the robustness of machine learning models [4], as seen in computer vision [5] and control tasks [6]. Although effective, this method is often time-consuming, data-intensive, and reduces solution traceability. The second approach uses real-world data to improve model accuracy [3] and is widely known as system identification [7] and dynamic modeling [8], [9]. However, in mostly used robotics physics simulators [10], such as Gazebo, simulation parameters are primarily manually tuned as the involvement of non-differentiable physics engines and model abstractions complicates the determination of parameter values. Consequently, aligning simulated behavior with real-world outcomes becomes a labor-intensive process that requires expert knowledge.

This work aims to apply data-driven simulation tuning methods to reduce reliance on expert knowledge and manual tuning for non-differentiable rigid body simulations. We propose a pipeline that compares motion trajectories of simulation models with real-world data and employs black-box optimization to automatically tune model parameters for wheeled mobile robots with varying kinematics. To ensure usability and relevance within the robotics community, we establish three design goals (DG):

DG1: Flexibility to integrate various optimization algorithms tailored to specific simulation models.
DG2: Applicability to wheeled mobile robots with diverse kinematics.
DG3: Ability to adjust a wide range of model parameters, making it adaptable to different robots and environments.

The primary contribution of this work is the adaption of data-driven approaches for the automated tuning of non-differentiable rigid body simulations. Our focus is on creating a broadly applicable methodology rather than solely optimizing the sim-to-real gap for a single robot. We explore the methodology and design goals through experiments with straight-line motions, utilizing four mobile robots that represent three distinct kinematic types, along with four optimization algorithms and multiple model parameters.

II. Related Work

There is a substantial amount of work that uses real-world data to enhance simulation models, with many employing system identification, a method for deriving models from measurements [7]. Real-world data is used in parameter identification to estimate specific characteristics of the system, such as friction, inertia, or dynamics parameters [11], [12]. However, many characteristics of robotic systems, such as tolerances or delays, are usually unknown or not accurately modeled. Therefore, a mathematical model is often developed that abstracts the overall system dynamics. In the area of wheeled mobile robots, particularly omnidirectional mobile robots, several works [8], [13] establish mathematical dynamic models to represent the input-output behavior of the real system. In [14], the challenges in modeling the real-world behavior of an omnidirectional robot are highlighted and an advanced model is developed for its sliding behavior with parameters estimated through real-world experiments.

This research has partly been funded by the Federal Ministry of Education and Research of Germany and the state of North-Rhine Westphalia as part of the Lamarr Institute for Machine Learning and Artificial Intelligence.

[1]M. Wiedemann, O. Ahmed, P. Detzner, and S. Kerner are and M. Hatwar was with the Fraunhofer Institute for Material Flow and Logistics, Dortmund, Germany.

[2] R. Gasoto is with NVIDIA, Santa Clara, CA, USA

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on July 31,2026 at 16:18:52 UTC from IEEE Xplore. Restrictions apply.

In [9] the omniwheeled DJI RoboMaster is modeled in Simulink with 23 parameters, which are subsequently tuned using real-world measurements and Evolutionary Algorithms (EAs) such as NSGA-II.

Nowadays, robots are often modeled in simulators using physics engines such as Bullet, Dynamic Animation and Robotics Toolkit (DART), or PhysX [10], [15]. Instead of mathematical modeling, these engines apply physical principles. Therefore, the model behavior needs to be tuned through various parameters using real-world data. For instance, [16] uses basic measurements to estimate mass, inertia, and motor characteristics to model a quadruped using PyBullet. [17] applies system identification to compare real-world trajectories of a humanoid robot with those generated in a DART-based simulation. In manipulation, [1] collects real-world end-effector trajectories of a manipulator and tunes the simulation model realized with different physics engines using EAs. [18] employs two derivative-free black-box optimization algorithms, Bayesian Optimization (BO) and Differential Evolution (DE), to align trajectories of a cable being manipulated in both simulation and reality.

Recent advances in differentiable physics engines [19], [20] offer the potential to optimize simulation parameters and reduce the sim-to-real gap through gradient-based optimization [21]. This potential is used to optimize simulation models for wheeled mobile robots with omnidirectional drives [22] and Ackerman steering [23]. However, the most widely used and feature-rich robotics physics simulators currently rely on non-differentiable physics engines [10].

To our knowledge, tuning models for wheeled mobile robots in these non-differentiable physics engine-based simulators predominantly relies on manual tuning and expert knowledge. This approach is often time-consuming and prone to errors, particularly due to the interdependencies among parameters. To address this challenge, we propose a pipeline for the automated tuning of wheeled mobile robot models by adapting optimization methods from mathematical modeling and other robotic applications.

III. Pipeline for Automated Model Tuning

In this section, we present our methodology for automating the tuning process for simulation models of wheeled mobile robots. In case of a perfect simulation, the outputs of the real system and its simulation model would perfectly align for the same input. However, in practice, discrepancies always arise due to the numerous abstractions present in simulations. Given the complexity of real robots, only the most important components necessary for functionality, such as wheels, the base, and sensors, are modeled, while their shapes and colliders are abstracted as rigid bodies. Furthermore, the physics engines used do not capture all the physical effects of the real world. Base controllers are developed based on the kinematics of wheeled mobile robots, but they do not account for all details of the onboard controllers. Actuators such as motors are replicated by joints, neglecting uncertainties such as gear ratios, delays, and tolerances. To address the sim-to-real gap caused by these abstractions and uncertainties, often simulation model parameters are adjusted based on expert knowledge to minimize discrepancies between input-output behaviors. Although simulation experts may understand how changes to specific parameters affect the model, determining the required value changes often relies on trial-and-error experiments due to the complexity of the simulation models and physics solvers. For automated tuning, we adapt the approach of changing simulation model parameters and comparing the behaviors of the real and simulated robots. Since the most widely used rigid body simulators in robotics are non-differentiable [19], and not all components of the real system are fully understood, we treat both the real robot and its model as black boxes. To compare the input-output behavior between simulation and reality, we incorporate the methodology of [24] into our pipeline architecture, as illustrated in Fig. 1. The modules we adopted are highlighted in white, while the new additions are marked in green.

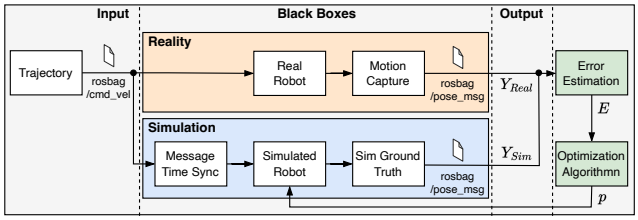


Fig. 1: Pipeline architecture extending the data collection of [24] towards an automated model tuning by adding a feedback loop containing the modules marked in green.

As **input** a trajectory of target velocities is sent to both the real robot and its simulation model. To ensure that the pipeline is agnostic to different robots (DG2), the Robot Operating System (ROS) is utilized for communication, as it is widely adopted in the robotics community and provides standardized interfaces. Furthermore, most robotics physics simulators support ROS [10]. All ROS messages in the pipeline, including trajectories, are saved as bag files to enable repeated playback and analysis. For simulation, a dedicated node manages time synchronization to ensure accurate message timing, compensating for variations in simulation speed relative to real-time.

The **output** consists of the kinematic data of the robot during motion. Although the pipeline architecture can handle pose and velocity data in both reality and simulation, this work focuses only on pose data. The pose data of the real system are denoted as $\mathbf{Y}_{Real}$, which is an array of n pose data points, where each element corresponds to a pose recorded during the robot's motion. Given that this study is limited to two-dimensional robots, each pose element includes the relative x- and y-position in meters, along with the orientation θ in radians.

$$\mathbf{Y}_{Real} = \left(\begin{pmatrix} x_1 \\ y_1 \\ \theta_1 \end{pmatrix}, \begin{pmatrix} x_2 \\ y_2 \\ \theta_2 \end{pmatrix} \cdots \begin{pmatrix} x_n \\ y_n \\ \theta_n \end{pmatrix} \right) \tag{1}$$

Pose data $\mathbf{Y}_{Real}$ are recorded using a Motion Capture (MoCap) system with submillimeter accuracy. Alternative

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on July 31,2026 at 16:18:52 UTC from IEEE Xplore. Restrictions apply.

localization techniques can also be utilized to obtain pose data. However, good real-world data are of great importance for successful tuning. A systematic error in the localization system can lead to poor training data and reinforce faulty system behavior during tuning.

Like $\mathbf{Y}_{Real}$ the pose data for the simulation model are defined as $\mathbf{Y}_{Sim}$, an array of poses over time. But in contrast to the real system, the simulation output can be modified by adjusting **model parameters**. These parameters are defined as a set $\mathbf{p} \in \mathbb{R}^l$ where each element p_i corresponds to one specific parameter of the l parameters used for tuning. According to DG3, a variety of model parameters are supported. Some parameters are applicable to all robots, while others are specific to a particular type or individual robot. Examples are discussed in Sec. IV-C. Based on [25] the output of the simulation $\mathbf{Y}_{Sim}$ is formulated as a result of the unknown internal model of the simulation $d(\mathbf{p})$ depending on the model parameters within (2).

$$\mathbf{Y}_{Sim} = d(\mathbf{p}) = \left(\begin{pmatrix} x_1 \\ y_1 \\ \theta_1 \end{pmatrix}, \begin{pmatrix} x_2 \\ y_2 \\ \theta_2 \end{pmatrix} \cdots \quad \begin{pmatrix} x_m \\ y_m \\ \theta_m \end{pmatrix} \right) \tag{2}$$

The pose data in $\mathbf{Y}_{Sim}$ is provided by the simulation and includes m pose data points of the model along the executed trajectory, where m depends on the duration of the trajectory and the publish rate of the simulation. This rate may vary slightly due to its dependence on simulation performance, which typically deviates over time.

The **Error Estimation** leverages output data from both systems to compute a sim-to-real error $\mathbf{E}(\mathbf{p})$. While previous works [1], [9] employed Euclidean Distance, our pipeline utilizes Dynamic Time Warping (DTW) [26], a technique widely used in speech recognition and even applied in robotics to compare manipulator trajectories [27]. DTW aligns time series of varying lengths by minimizing the cumulative distance and is implemented using the DTAIDistance library [28]. This improves the flexibility of the pipeline, as it allows a mismatch in the number of data points between $\mathbf{Y}_{Real}$ and $\mathbf{Y}_{Sim}$ when $n \neq m$. Furthermore, the computation of separate errors for each kinematic characteristic of the robot is possible. For example, we can calculate the error specifically for the pose in the x-direction. This allows us to adjust the sim-to-real gap based on different characteristics, simplifying the transfer of our methodology to other robot types, including those operating in three-dimensional space. As we focus on planar motion, errors are computed for the pose data in x, y and orientation θ, resulting in the following error vector:

$$\mathbf{E}(\mathbf{p}) = DTW(\mathbf{Y}_{Sim}(\mathbf{p}), \mathbf{Y}_{Real}) = \begin{pmatrix} E_x \\ E_y \\ E_\theta \end{pmatrix} \tag{3}$$

To account for potential random errors in the real system, the module is designed to incorporate multiple real-world measurements for each trajectory by averaging the data.

Optimization Algorithm is the next step, as the pipeline aims to reduce the sim-to-real error during the automated model tuning process by searching for the model parameters that result in the lowest error. Hence, the following black-box optimization problem [25] is formulated:

$$\min_{\mathbf{p}} f(\mathbf{E}(\mathbf{p})) \tag{4}$$

$$p_i \in \left[P_i^L, P_i^U\right], \quad \text{for } i \in \{1, 2, \ldots, l\} \tag{5}$$

Within (4), the objective function to be minimized is based on the sim-to-real error $\mathbf{E}(\mathbf{p})$. The model parameters are considered independent decision variables under the constraint of the lower and upper bounds given for each parameter in (5). Within the module, several algorithms and fitness functions can be implemented to solve the black-box optimization problem, following DG1. Based on insights from the related work in Sec. II and DG3, we do not use Bayesian Optimization (BO), which has limitations on the number of tunable decision variables [1]. Instead, this work focuses on Evolutionary Algorithm (EA). During the evolutionary optimization process, the EA generates individuals for the model parameters $\mathbf{p}$ with the aim of finding a candidate that achieves the lowest value for the objective function. For each candidate in a population, the parameters are applied to the simulation, and the optimization module triggers a repetition of the trajectory for the simulation model. This procedure is repeated until a certain termination criterion is met, and then the best-found candidate set of parameters is the solution of the optimization problem.

IV. Experimental Setup

In the following, we present the experimental setup, including the used optimization algorithms, wheeled mobile robots, and model parameters.

A. Optimization Algorithms

Following DG1, the flexibility to implement various optimization algorithms is evaluated by implementing and testing the following EAs in our experiments:

- Genetic Algorithms (GA) [29]
- Particle Swarm Optimization (PSO) [30]
- Differential Evolution (DE) [31]
- Evolutionary Strategy (ES) [32]

All algorithms are implemented using the pymoo framework [33], which supports single- and multi-objective optimization. The algorithms used are single-objective, requiring a fitness function f to be defined for (4), which accounts for the sim-to-real error $\mathbf{E}(\mathbf{p})$ as given in (3). In our experiments, the fitness function (6) computes the weighted sum of $\mathbf{E}(\mathbf{p})$.

$$f(\mathbf{E}(\mathbf{p})) = \mathbf{W} \cdot \mathbf{E}(\mathbf{p}) = \mathrm{diag}(1, 1, 0.57) \cdot \mathbf{E}(\mathbf{p}) \tag{6}$$

The weights in the fitness function can be selected individually and were determined through preliminary tests involving various robots and trajectories. Consequently, as given in (6), x and y are assigned equal weights. Experiments revealed that assigning a higher weight to one direction led to an increase in overall deviation. The orientation θ is assigned a smaller weight to prioritize reducing the error in x and y. For replicating rotational or sliding effects, a larger weight for θ may be advantageous.

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on July 31,2026 at 16:18:52 UTC from IEEE Xplore. Restrictions apply.

B. Wheeled Mobile Robots

According to DG2, the applicability to various wheeled mobile robots is tested using four robots with three kinematics, listed in Table I and shown in Fig. 2.

TABLE I: Overview of used robots and their properties.

#	Name	Kinematic	Velocity
1	Robotnik RB-1 Base	Differential*	$v_{max} = 1.55\,m/s$
2	AgileX Hunter SE	Ackermann	$v_{max} = 4.8\,m/s$
3	DJI RoboMaster S1	Omnidirectional	$v_{X,max} = 4\,m/s$ $v_{Y,max} = 3\,m/s$
4	O^{3}dyn	Omnidirectional	$v_{X,max} = 10\,m/s$ $v_{Y,max} = 10\,m/s$

* omniwheels as caster wheel replacement

The Robotnik RB-1 Base, the AgileX Hunter SE, and DJI RoboMaster S1 are commercial platforms. Notably, the RoboMaster is a modified version equipped with an NVIDIA Jetson board, additional sensors, and a redesigned software stack for communication via ROS. O^{3}dyn [34] is a logistics robot developed at Fraunhofer IML, designed for highly dynamic pallet transport indoors and outdoors.

The robots are modeled in Isaac Sim by importing construction data in Computer Aided Design (CAD) or Unified Robot Description Format (URDF) format. As commonly done in robotics, the joints and wheels are modeled to best represent driving behavior while abstracting most of the other components to maintain computational efficiency and reduce modeling effort. Mass values and inertia parameters are imported or defined on the basis of vendor data. To enable robot's motion, the robot's base controllers are replicated using Action Graphs in Isaac Sim.

The active wheels of the RB-1 Base and Hunter are approximated as spheres to ensure smooth motion with minimal computational effort. The modeling of omnidirectional platforms is more challenging due to the Mecanum wheels, which consist of multiple free-spinning rollers. Following [24], spherical collider models are used to represent the rollers. O^{3}dyn is the most complex robot in these experiments as it features besides Mecanum wheels an air suspension system that introduces damping behavior and varying mass distribution during operation.

The robot's driving performance is tested once the import and initial modeling of the robots are complete, and a functional base controller is implemented. By observing the robot's motion in the simulation and adjusting its behavior through a comparison of commanded and actual velocities, the functionality of the models is ensured. This version of the simulation models is hereafter referred to as the baseline and serves as the starting point for the tuning experiments.

Within the experimental evaluation, the robots are tested by performing various trajectories, listed in Table II. Only the omnidirectional robots can move laterally, and the RB-1 is limited to x_1_1_20 due to its small maximum velocity. To balance the number of experiments while including different kinematics, our work focuses on straight-line motion in the x- and y-directions. For more in-depth investigations into how much we can reduce the sim-to-real gap for a robot, further studies involving more complex trajectories, including rotational motions, are necessary. However, this is not the focus of this work, which emphasizes general applicability across different robotic platforms.

TABLE II: Trajectories used in the experiments.

Name	a_X	$v_{X,max}$	a_Y	$v_{Y,max}$	Length
x_1_1_20	1 m/s²	1 m/s	0 m/s²	0 m/s	20 m
x_2_2_20	2 m/s²	2 m/s	0 m/s²	0 m/s	20 m
x_3_3_20	3 m/s²	3 m/s	0 m/s²	0 m/s	20 m
y_1_1_20	0 m/s²	0 m/s	1 m/s²	1 m/s	20 m
y_2_2_20	0 m/s²	0 m/s	2 m/s²	2 m/s	20 m
y_3_3_10	0 m/s²	0 m/s	3 m/s²	3 m/s	10 m

All trajectories follow a trapezoidal velocity profile. Initially, the robot accelerates at a rate of a until it reaches its maximum velocity v_{max}. Both acceleration and maximum velocity are deliberately set to values that remain within the limits of the onboard controller. Once at v_{max}, the robot maintains a constant speed before decelerating at a rate of $-a$. The duration of the constant velocity phase is designed to ensure that the trajectory covers a fixed distance. To ensure safety during real-world experiments, the maximum velocity is limited to $v_{max} = 3\,m/s$, and the trajectory length is reduced for the fastest lateral motion. To compensate for random errors, each trajectory is executed ten times during the testing. For other robots with greater variation in motion, more repetitions may be needed; however, this number has proven sufficient for our experiments.

C. Model Parameters

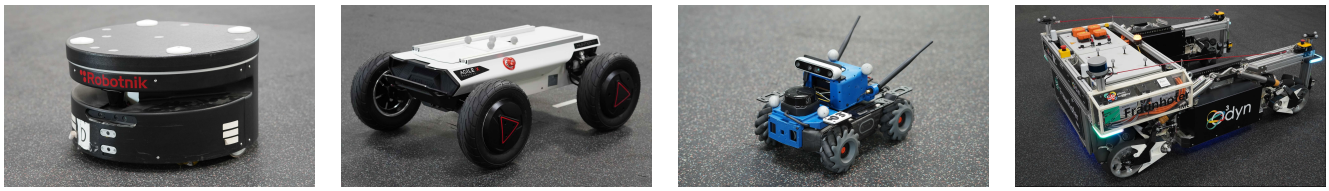
DG3 is tested using various model parameters, the decision variables $\mathbf{p}$ in our black-box optimization. The parameters used in this work, along with their value ranges (5), are listed in Table III. Some parameters are applicable to all robots, others specific to robot types or models. For example, friction parameters (p_1, p_2) are relevant for all robots, while O^{3}dyn

TABLE III: Overview of the used model parameters and their value ranges $[P^L, P^U]$ for various robots.

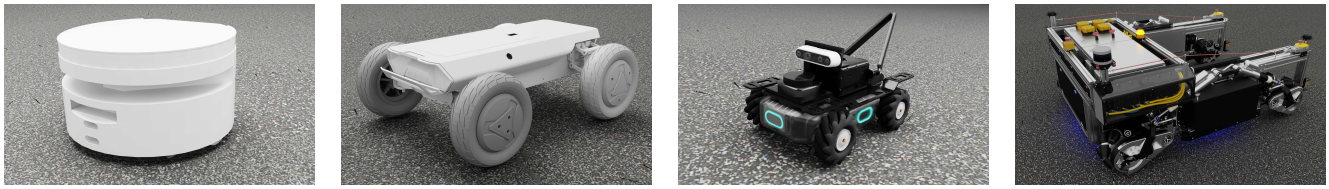
Name	Parameter	Unit	Default	$[P^L, P^U]$
p_1	static friction	-	1.0	[0.1, 1.0]
p_2	dynamic friction	-	1.0	[0.1, 1.0]
$p_3 - p_5$	center of mass $x/y/z$	m	$\star$	$\star$
$p_6 - p_8$	diagonal inertia	$kg{\cdot}m^2$	$\star$	$\star$
$p_9 - p_{12}$	principal axes	-	0.0	[-1.0, 1.0]
p_{13}	motor joint stiffness$\star\star$	$\frac{kg}{s^2}$	100	[0, 1000]
p_{14}	motor joint damping$\star\star$	$\frac{kg}{s}$	10	[0, 100]

$\star$ robot-specific values; $\star\star$ only relevant for O^{3}dyn.

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on July 31,2026 at 16:18:52 UTC from IEEE Xplore. Restrictions apply.



(a) Robotnik RB-1 Base in reality (b) AgileX Hunter SE in reality (c) DJI RoboMaster S1 in reality (d) O^{3}dyn in reality



(e) Robotnik RB-1 Base in simulation (f) AgileX Hunter SE simulation (g) DJI RoboMaster S1 in simulation (h) O^{3}dyn in simulation

Fig. 2: Robots used for experiments: the differential drive Robotnik RB-1 Base (2(a), 2(e)), AgileX Hunter SE with Ackermann steering (2(b), 2(f)), and two omnidirectional robots DJI RoboMaster S1 (2(c), 2(g)) and O^{3}dyn (2(d), 2(h)).

uniquely features the motor joints parameters (p_{13}, p_{14}) related to its damping system. The list of parameters within these experiments is not exhaustive, but includes the parameters identified as important in [24] and by initial tests. During the experiments, different subsets of these parameters are adjusted. Parameters which are typically known or easy to determine in reality, such as the overall mass, are set based on the real-world values.

D. Experiments

By combining the variations in robots, trajectories, algorithms, and model parameters, experiments (E) are carried out for each design goal, summarized as E1 to E3 in Table IV. For DG1, the optimization algorithms are tested on a simple tuning task with few parameters on a less dynamic robot and trajectory, as well as on a more complex tuning task with multiple parameters on a dynamic robot and trajectory (E1). To validate DG2, the pipeline is evaluated across all robots and trajectories using a single EA and five model parameters (E2). For DG3, the effect of varying the number of model parameters is tested in the most complex scenario, which includes O^{3}dyn and the fastest lateral trajectory (E3).

TABLE IV: Experiments per design goal.

DG	E	Algorithm	Robots	Trajectory	Parameter	n_gen*
1	1	All	RB-1 O^{3}dyn	x_1_1_20 y_3_3_10	5p: $p_1 - p_5$ 14p: $p_1 - p_{14}$	40 40
2	2	GA	All	All	5p: $p_1 - p_5$	40
3	3	GA	O^{3}dyn	y_3_3_10	3p: $p_3 - p_5$ 5p: $p_1 - p_5$ 10P: $p_3 - p_{12}$ 14p: $p_1 - p_{14}$	100 100 100 100

* number of generations with a population size of 40

V. Results & Discussion

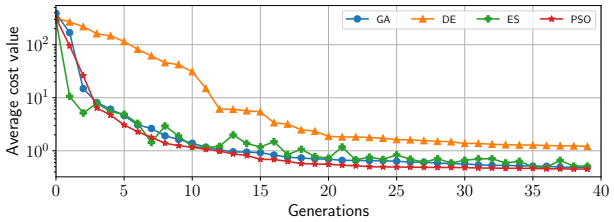
In this section, we present the results for the experiments defined in Sec. IV-D and summarized in Tab. IV.

A. Optimization Algorithms

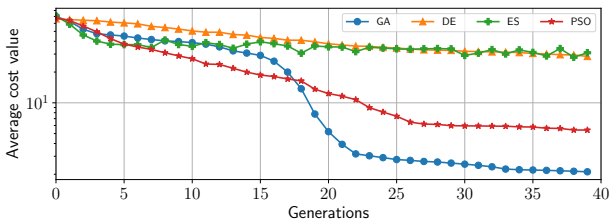
E1 investigates the applicability of optimization algorithms for both simple and complex tuning tasks. The results, presented on a logarithmic scale in Fig. 3, show the average cost per generation for the EAs discussed in Sec. IV-A.

In the simple case (Fig. 3(a)), most algorithms converge quickly, exhibiting similar speeds and comparable final cost values, with only DE converging slower and resulting in higher average costs. In the challenging case (Fig. 3(b)), the initial cost values are lower because the trajectory is only half as long. Although all algorithms reduce the average costs during tuning, the improvements are smaller due to the increased task complexity. Notably, PSO and particularly GA outperform the other algorithms.

Overall, the results of E1 demonstrate that the pipeline



(a) Simple tuning task: RB-1, x_1_1_20, 5 parameter (p1 - p5).



(b) Complex tuning task: O^{3}dyn, y_3_3_10, 14 parameters (p1 - p14).

Fig. 3: Average cost per generation of EAs from Sec. IV-A for tuning tasks of varying complexity.

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on July 31,2026 at 16:18:52 UTC from IEEE Xplore. Restrictions apply.

meets DG1, as all optimization algorithms reduce the sim-to-real error for the tuning tasks. Due to its strong performance, GA is utilized in subsequent experiments.

B. Robots and Trajectories

In E2, the pipeline is evaluated on all wheeled robots and trajectories. The results are given in Table V, which lists the total error for both the baseline and tuned models, along with the error reduction. The sum of E_x, E_y, E_θ from (3) is used to exclude the weighting of the fitness function (6). It is important to note that the total error is affected by both the trajectory length and the number of output data points. In particular, a deviation between the real robot and the simulation model can lead to a significantly higher error value if the trajectory length is higher. To complement the error values in Table V, a visual representation of the tuning effect in selected trajectories for the pose data of x and y is provided in Fig. 4. while the graphs do not consider the orientation of the robot. Thus, a low error value does not necessarily indicate perfectly matching plots.

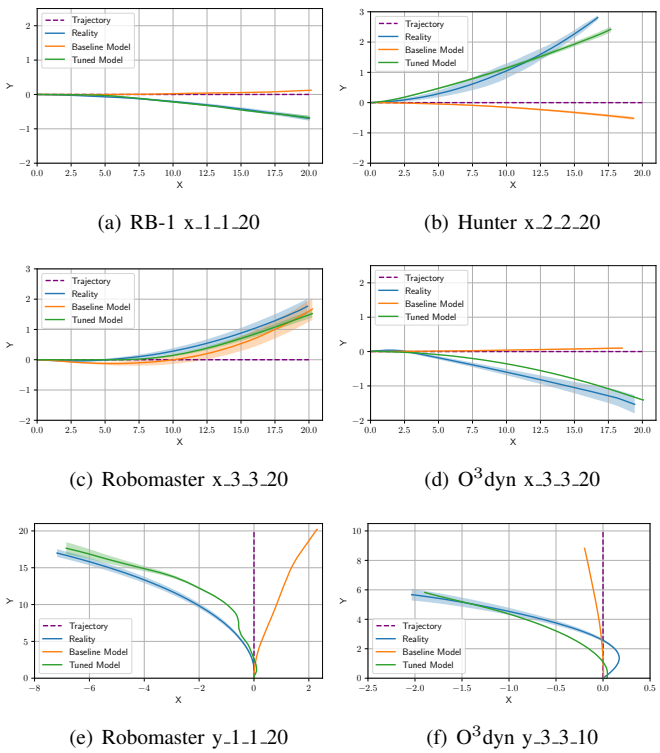
Despite tuning only five model parameters ($p_1 - p_5$) over 40 generations, significant improvements are observed across all robots and trajectories in Table V. It is important to note that the model parameters of the baseline models are not set randomly. Rather, they are set by prior knowledge during the modeling process, as discussed in Sec. IV-B.

For the differential robot RB-1, the error is reduced by 67.5 %, a moderate gain compared with the other experiments. Because the initial model was already accurate, tuning yields an almost perfect match to reality (see Fig. 4(a)).

TABLE V: Comparison of sim-to-real error before and after model tuning for all robots and trajectories.

Robot	Trajectory	Total Error*		
		Baseline	Tuned	Reduction
RB-1	x_1_1_20	16.84	5.47	67.5 %
Hunter	x_1_1_20	95.3	13.32	86.0 %
Hunter	x_2_2_20	76.1	3.08	95.9 %
Hunter	x_3_3_20	88.39	1.20	98.6 %
RoboMaster	x_1_1_20	17.39	5.90	66.0 %
RoboMaster	x_2_2_20	25.33	5.29	79.1 %
RoboMaster	x_3_3_20	5.08	3.30	35.0 %
RoboMaster	y_1_1_20	294.8	56.4	80.9 %
RoboMaster	y_2_2_20	227.67	36.17	84.1 %
RoboMaster	y_3_3_10	48.06	6.09	87.3 %
O^{3}dyn	x_1_1_20	11.71	4.48	61.7 %
O^{3}dyn	x_2_2_20	30.45	6.71	78.0 %
O^{3}dyn	x_3_3_20	41.08	5.30	87.1 %
O^{3}dyn	y_1_1_20	105.1	48.67	53.7 %
O^{3}dyn	y_2_2_20	154.4	42.4	72.5 %
O^{3}dyn	y_3_3_10	80.9	1.95	97.5 %

* Based on (3): Total Error = E_x[m] + E_y[m] + E_θ[rad]



(a) RB-1 x_1_1_20 (b) Hunter x_2_2_20

(c) Robomaster x_3_3_20 (d) O^{3}dyn x_3_3_20

(e) Robomaster y_1_1_20 (f) O^{3}dyn y_3_3_10

Fig. 4: Comparison of robot trajectories for the physical robot, the baseline, and the tuned model across several motions. Each plot shows the mean (solid line) and standard deviation (shaded) from ten open-loop runs. The lack of feedback control allows drift to accumulate, causing deviations between the commanded and actual motion, especially for lateral trajectories.

The same reason explains the overall smallest improvement of 35.0 % for the RoboMaster at x_3_3_20; nevertheless, Fig. 4(c) shows that the simulation closely replicates real behavior. The Hunter tuning (Ackermann steering) achieves the highest average reduction, reaching 98.6 % for x_3_3_20. These improvements are underlined in Fig. 4(b), showing how the y-drift has been effectively shifted from the baseline to the tuned model to better match real-world behavior. The results for omnidirectional robots suggest that the magnitude of improvement increases with the complexity of the robot's motion: error reductions are smaller for slower longitudinal motions (RoboMaster: 66.0 %, O^{3}dyn: 61.7 %) than for dynamic lateral motion (RoboMaster: 87.3 %, O^{3}dyn: 97.5 %). The pattern likely reflects the stronger influence of modeling errors during dynamic maneuvers. Lateral motion, in particular, is harder to replicate accurately when building the model. Therefore, parameter optimization yields higher error reductions, even though the alignment between simulation and real-world trajectories appears less precise in the resulting graphs (Fig. 4(e), 4(f)) than in the slower longitudinal case.

Overall, the results for E2 demonstrate that the proposed pipeline fulfills DG2 since the sim-to-real error is reduced for all wheeled mobile robots and trajectories.

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on July 31,2026 at 16:18:52 UTC from IEEE Xplore. Restrictions apply.

C. Model Parameters

Experiment E3 examines how the number of tuned parameters affects the accuracy of the model, the resulting trajectories are shown in Fig. 5. In reality, the commanded 10 m straight lateral motion produces a shorter path and significant drift along the x-axis. The baseline simulation model reproduces this only weakly. Optimizing only the center of mass (3p) reduces the sim-to-real error, but it remains large. The pronounced deviation in the x-direction indicates a considerable slip, so adding friction parameters into the training (5p) greatly narrows the gap. As a cross-check we replaced the friction parameters with inertia and the motor joints for tuning O^{3}dyn's damping behavior (10p). The improvement is smaller even if more parameters are used: inertia mainly influences rotation, which is slightly relevant for this lateral path, and O^{3}dyn's damping seems to have little effect here. Adjustment of all parameters (14p) provides only marginal gains over the (5p) model, highlighting the dominant role of friction in reproducing the slip of O^{3}dyn. This also suggests that the default inertia and damping settings are adequate for this scenario, although tests with trajectories with significant rotation are needed to check the influence of these parameters and avoid overfitting.

In conclusion, targeting a few high-impact parameters often ensures better results than tuning many less consequential ones. Broader studies are required, including a wider range of mobile robots, models from multiple developers, and diverse trajectories to develop robust best-practice guidelines and identify recurring modeling challenges.

However, a comparison of the trajectory between the real robot and the best model reveals that not all systematic errors of O^{3}dyn are captured. The real robot initially moves in the positive x-direction, which is then overlaid by a systematic negative x-drift. Although the tuned model replicates this drift, it lacks the initial forward push. This behavior requires further investigation into the dynamics of the system, modeling, and tuning parameters. However, such analysis is beyond the scope of this work, which focuses on the methodology of the pipeline. Meanwhile, the implementation of the pipeline based on our design goals allows the easy integration of solutions to address these inaccuracies, such as actuator parameter tuning and the use of velocity data through multi-objective optimization algorithms.

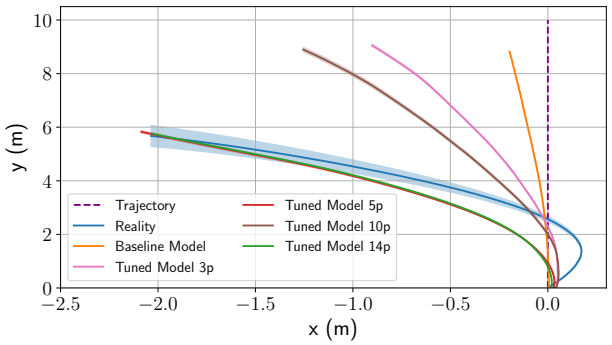


Fig. 5: Comparison of O^{3}dyn's lateral trajectory y_3_3_10 for models tuned with varying number of parameters. Each plot shows the mean (solid line) and standard deviation (shaded) from ten trials.

Overall, the results of E3 demonstrate that the pipeline meets DG3 and can adjust multiple parameters, including those that are typically difficult to estimate, such as diagonal inertia or principal axes. This reduces modeling efforts and challenges. However, increasing the number of parameters also increases the number of generations required, thereby extending the tuning time.

VI. Conclusion

In this work, we employed black-box optimization within a pipeline to automatically tune non-differentiable, rigid body simulation models of wheeled mobile robots. The method reduces the dependence on expert knowledge and manual effort while narrowing the sim-to-real gap. It is driven by three design goals: (DG1) flexibility in the choice of optimization algorithms, (DG2) applicability across different wheeled mobile robots, and (DG3) simultaneous tuning of multiple model parameters. After issuing identical velocity commands, the pipeline records motion data from both the real robot and its simulated counterpart. These data are then used for black-box optimization that adjusts model parameters to minimize discrepancies in output behavior. We conducted an experimental evaluation to validate the pipeline and its design goals, involving four mobile robots with three kinematics (differential, Ackermann, and omnidirectional), four EAs (GA, DE, ES, PSO), and multiple sets of model parameters. For all straight-line trajectories, the pipeline reduced the sim-to-real gap across various combinations of robots, kinematics, algorithms, and model parameters.

Our approach shows potential for advancing robotics development through automated model tuning, addressing the challenges of the sim-to-real gap and the extensive effort required for robot modeling. However, the experiments were restricted to straight motions, yielding one parameter set per trajectory and risking overfitting.

Future work could gather data on varying accelerations and rotations and extend the training to multiple trajectories. This would help the pipeline determine a more general parameter set, which reduces the sim-to-real gap across diverse motions. However, given the robots' complexity, applied model abstractions during modeling, and the use of non-differentiable physics engines, it remains uncertain whether a universal parameter set is achievable. Reducing the abstractions of the model by incorporating more real-world components may help. For example, instead of using Isaac Sim's default base controller, one could integrate the actual robot controller. Another research direction is to switch between previously tuned parameter sets online to match various commanded motions.

Acknowledgment

We thank Anna Dieckhöfer and Ece Erdim for their help with robot modeling. ChatGPT is utilized for minor edits to the authors' text, including spelling and grammar corrections.

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on July 31,2026 at 16:18:52 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] J. Collins, R. Brown, J. Leitner, and D. Howard. Traversing the Reality Gap via Simulator Tuning. [Online]. Available: http://arxiv.org/abs/2003.01369

[2] N. Jakobi, P. Husbands, and I. Harvey, "Noise and the reality gap: The use of simulation in evolutionary robotics," in Advances in Artificial Life, F. Morán, A. Moreno, J. J. Merelo, and P. Chacón, Eds. Springer, pp. 704–720.

[3] S. Höfer, K. Bekris, A. Handa, J. C. Gamboa, F. Golemo, M. Mozifian, C. Atkeson, D. Fox, K. Goldberg, J. Leonard, C. K. Liu, J. Peters, S. Song, P. Welinder, and M. White. Perspectives on Sim2Real Transfer for Robotics: A Summary of the R:SS 2020 Workshop. [Online]. Available: http://arxiv.org/abs/2012.03806

[4] S. Höfer, K. Bekris, A. Handa, J. C. Gamboa, M. Mozifian, F. Golemo, C. Atkeson, D. Fox, K. Goldberg, J. Leonard, C. Karen Liu, J. Peters, S. Song, P. Welinder, and M. White, "Sim2Real in Robotics and Automation: Applications and Challenges," vol. 18, no. 2, pp. 398–400.

[5] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 23–30. [Online]. Available: https://ieeexplore.ieee.org/document/8202133

[6] Y. Chebotar, A. Handa, V. Makoviychuk, M. Macklin, J. Issac, N. Ratliff, and D. Fox, "Closing the Sim-to-Real Loop: Adapting Simulation Randomization with Real World Experience," in 2019 International Conference on Robotics and Automation (ICRA), pp. 8973–8979. [Online]. Available: https://ieeexplore.ieee.org/document/8793789

[7] J. Hollerbach, W. Khalil, and M. Gautier, "Model Identification," in Springer Handbook of Robotics, B. Siciliano and O. Khatib, Eds. Springer, pp. 321–344. [Online]. Available: https://doi.org/10.1007/978-3-540-30301-5_15

[8] N. Tlale and M. Villiers, "Kinematics and Dynamics Modelling of a Mecanum Wheeled Mobile Platform," in 2008 15th International Conference on Mechatronics and Machine Vision in Practice, pp. 657–662. [Online]. Available: https://ieeexplore.ieee.org/document/4749608

[9] A. Kanwischer and O. Urbann, "A Machine Learning Approach to Minimization of the Sim-To-Real Gap via Precise Dynamics Modeling of a Fast Moving Robot," in 2022 17th International Conference on Control, Automation, Robotics and Vision (ICARCV), pp. 349–354. [Online]. Available: https://ieeexplore.ieee.org/document/10004376

[10] J. Collins, S. Chand, A. Vanderkop, and D. Howard, "A Review of Physics Simulators for Robotic Applications," vol. 9, pp. 51 416–51 431.

[11] M. Gautier and W. Khalil, "On the identification of the inertial parameters of robots," in Proceedings of the 27th IEEE Conference on Decision and Control, pp. 2264–2269 vol.3. [Online]. Available: https://ieeexplore.ieee.org/document/194738

[12] J. Wu, J. Wang, and Z. You, "An overview of dynamic parameter identification of robots," vol. 26, no. 5, pp. 414–419. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0736584510000232

[13] Z. Hendzel and Rykała, "Modelling of Dynamics of a Wheeled Mobile Robot with Mecanum Wheels with the use of Lagrange Equations of the Second Kind," vol. 22, no. 1, pp. 81–99.

[14] R. Williams, B. Carter, P. Gallina, and G. Rosati, "Dynamic model with slip for wheeled omnidirectional robots," vol. 18, no. 3, pp. 285–293. [Online]. Available: https://ieeexplore.ieee.org/document/1019459

[15] C. K. Liu and D. Negrut, "The Role of Physics-Based Simulators in Robotics," vol. 4, no. 1, pp. 35–58. [Online]. Available: https://www.annualreviews.org/doi/10.1146/annurev-control-072220-093055

[16] J. Tan, T. Zhang, E. Coumans, A. Iscen, Y. Bai, D. Hafner, S. Bohez, and V. Vanhoucke, "Sim-to-Real: Learning Agile Locomotion For Quadruped Robots," in Robotics: Science and Systems XIV. Robotics: Science and Systems Foundation. [Online]. Available: http://www.roboticsproceedings.org/rss14/p10.pdf

[17] J. Tan, Z. Xie, B. Boots, and C. K. Liu, "Simulation-based design of dynamic controllers for humanoid balancing," in 2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 2729–2736. [Online]. Available: https://ieeexplore.ieee.org/document/7759424

[18] V. Lim, H. Huang, L. Y. Chen, J. Wang, J. Ichnowski, D. Seita, M. Laskey, and K. Goldberg, "Real2Sim2Real: Self-Supervised Learning of Physical Single-Step Dynamic Actions for Planar Robot Casting," in 2022 International Conference on Robotics and Automation (ICRA), pp. 8282–8289.

[19] J. Degrave, M. Hermans, J. Dambre, and F. Wyffels, "A Differentiable Physics Engine for Deep Learning in Robotics," vol. 13. [Online]. Available: https://www.frontiersin.org/journals/neurorobotics/articles/10.3389/fnbot.2019.00006/full

[20] K. Werling, D. Omens, J. Lee, I. Exarchos, and C. K. Liu. Fast and Feature-Complete Differentiable Physics for Articulated Rigid Bodies with Contact. [Online]. Available: http://arxiv.org/abs/2103.16021

[21] J. Collins, R. Brown, J. Leitner, and D. Howard. Follow the Gradient: Crossing the Reality Gap using Differentiable Physics (RealityGrad). [Online]. Available: http://arxiv.org/abs/2109.04674

[22] E. Granados, A. Boularias, K. Bekris, and M. Aanjaneya, "Model Identification and Control of a Low-cost Mobile Robot with Omnidirectional Wheels using Differentiable Physics," in 2022 International Conference on Robotics and Automation (ICRA), pp. 1358–1364. [Online]. Available: https://ieeexplore.ieee.org/document/9812454

[23] B. M. Gonultas, P. Mukherjee, O. G. Poyrazoglu, and V. Isler, "System Identification and Control of Front-Steered Ackermann Vehicles Through Differentiable Physics," in 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 4347–4353. [Online]. Available: https://ieeexplore.ieee.org/document/10342391

[24] M. Wiedemann, O. Ahmed, A. Dieckhöfer, R. Gasoto, and S. Kerner, "Simulation Modeling of Highly Dynamic Omnidirectional Mobile Robots Based on Real-World Data," in 2024 IEEE International Conference on Robotics and Automation (ICRA), pp. 16 923–16 929. [Online]. Available: https://ieeexplore.ieee.org/document/10611459

[25] I. Bajaj, A. Arora, and M. M. F. Hasan, "Black-Box Optimization: Methods and Applications," in Black Box Optimization, Machine Learning, and No-Free Lunch Theorems, P. M. Pardalos, V. Rasskazova, and M. N. Vrahatis, Eds. Springer International Publishing, pp. 35–65. [Online]. Available: https://doi.org/10.1007/978-3-030-66515-9_2

[26] P. Senin, "Dynamic time warping algorithm review."

[27] B. Johnen and B. Kuhlenkoetter, "A Dynamic Time Warping algorithm for industrial robot motion analysis," in 2016 Annual Conference on Information Science and Systems (CISS), pp. 18–23. [Online]. Available: https://ieeexplore.ieee.org/document/7460470

[28] W. Meert, K. Hendrickx, T. Van Craenendonck, P. Robberechts, H. Blockeel, and J. Davis, "DTAIDistance," Zenodo. [Online]. Available: https://zenodo.org/records/7158824

[29] M. Kumar, D. M. Husain, N. Upreti, and D. Gupta. Genetic Algorithm: Review and Application. [Online]. Available: https://papers.ssrn.com/abstract=3529843

[30] J. Kennedy and R. Eberhart, "Particle swarm optimization," in Proceedings of ICNN'95 - International Conference on Neural Networks, vol. 4, pp. 1942–1948 vol.4. [Online]. Available: https://ieeexplore.ieee.org/document/488968

[31] K. Price, R. Storn, and J. Lampinen, Differential Evolution-A Practical Approach to Global Optimization, vol. 141.

[32] H.-G. Beyer and D. V. Arnold, "Theory of Evolution Strategies — A Tutorial," in Theoretical Aspects of Evolutionary Computing, L. Kallel, B. Naudts, and A. Rogers, Eds. Springer, pp. 109–133. [Online]. Available: https://doi.org/10.1007/978-3-662-04448-3_6

[33] J. Blank and K. Deb, "Pymoo: Multi-Objective Optimization in Python," vol. 8, pp. 89 497–89 509. [Online]. Available: https://ieeexplore.ieee.org/document/9078759

[34] N. Ullrich and M. Gössner, "Analyse und optimierung eines ftf-fahrwerks mit mecanum-rädern," p. Issue 18. [Online]. Available: https://www.logistics-journal.de/proceedings/2022/5606

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on July 31,2026 at 16:18:52 UTC from IEEE Xplore. Restrictions apply.